
Experiment 7

Continuous Power Flow in MATLAB – Stable Region

Lab Report

Student Name HAMZA AHMAD

Registration Number 2022-EE-1654

Section B

Power System Operation and Control Laboratory

Fall 2025

MATLAB code for CPF implementation

```
clear; clc; close all;

fprintf('\n==== CONTINUOUS POWER FLOW (CPF) - IEEE-14 BUS ==== \n\n');

BMva = 100;

% Columns: Bus | Vsp | delta | Pg | Qg | Pl | Ql | Qmin | Qmax | Type
busdata = [
    1  1.060  0  232.4  -16.9  0.0  0.0  0  0  1;
    2  1.045  0  40.0  43.56  21.7  12.7  -40  50  2;
    3  1.010  0  0.0  0.0  94.2  19.0  -25  40  3;
    4  1.000  0  0.0  0.0  47.8  -3.9  0  0  3;
    5  1.000  0  0.0  0.0  7.6  1.6  0  0  3;
    6  1.070  0  0.0  0.0  11.2  7.5  -6  24  2;
```

```

7   1.000   0       0.0   0.0   0.0   0.0   0   0   3;
8   1.090   0       0.0   0.0   0.0   0.0   0   0   2;
9   1.000   0       0.0   0.0  29.5  16.6   0   0   3;
10  1.000   0       0.0   0.0   9.0   5.8   0   0   3;
11  1.000   0       0.0   0.0   3.5   1.8   0   0   3;
12  1.000   0       0.0   0.0   6.1   1.6   0   0   3;
13  1.000   0       0.0   0.0  13.5   5.8   0   0   3;
14  1.000   0       0.0   0.0  14.9   5.0   0   0   3;

```

```
];
```

```
% Line Data
```

```
linedata = [
```

```

1   2   0.01938   0.05917   0.0264   1.000;
1   5   0.05403   0.22304   0.0246   1.000;
2   3   0.04699   0.19797   0.0219   1.000;
2   4   0.05811   0.17632   0.0170   1.000;
2   5   0.05695   0.17388   0.0173   1.000;
3   4   0.06701   0.17103   0.0064   1.000;
4   5   0.01335   0.04211   0.0000   1.000;
4   7   0.00000   0.20912   0.0000   0.978;
4   9   0.00000   0.55618   0.0000   0.969;
5   6   0.00000   0.25202   0.0000   1.000;
6  11   0.09498   0.19890   0.0000   1.000;
6  12   0.12291   0.25581   0.0000   1.000;
6  13   0.06615   0.13027   0.0000   1.000;
7   8   0.00000   0.17615   0.0000   1.000;
7   9   0.00000   0.11001   0.0000   1.000;
9  10   0.03181   0.08450   0.0000   1.000;
9  14   0.12711   0.27038   0.0000   1.000;
10  11   0.08205   0.19207   0.0000   1.000;
12  13   0.22092   0.19988   0.0000   1.000;
13  14   0.17093   0.34802   0.0000   1.000;

```

```
];
```

```
%% Y-BUS FORMATION
```

```

nbus = max(max(linedata(:, 1:2)));

Y = zeros(nbus, nbus);

fb = linedata(:, 1);
tb = linedata(:, 2);

for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);

    R = linedata(k, 3);
    X = linedata(k, 4);
    Bc = linedata(k, 5);
    tap = linedata(k, 6);
    if tap == 0, tap = 1; end

    Z = R + 1i * X;
    y = 1 / Z;
    ysh = 1i * Bc;

    Y(i, j) = Y(i, j) - y / tap;
    Y(j, i) = Y(i, j);
    Y(i, i) = Y(i, i) + y / (tap^2) + ysh;
    Y(j, j) = Y(j, j) + y + ysh;
end

G = real(Y);
B = imag(Y);

type = busdata(:, 10);
V0 = busdata(:, 2);
del0 = deg2rad(busdata(:, 3));
Pg0 = busdata(:, 4) / BMva;
Qg0 = busdata(:, 5) / BMva;
Pl0 = busdata(:, 6) / BMva;
Ql0 = busdata(:, 7) / BMva;
pq = find(type == 3);
pv = find(type == 2);

```

```

npq = length(pq);
npv = length(pv);
lambda = 0;          lambda_max = 5.0;
d_lambda = 0.05;      % Initial step size
tol = 1e-6;          max_iter = 20;
lambda_trace = [];
V_trace = [];
V = V0;
del = del0;
Pl_dir = Pl0;
Ql_dir = Ql0;
fprintf('  lambda range: [0, %.2f]\n', lambda_max);
fprintf('  initial step: %.4f\n', d_lambda);
fprintf('  tolerance: %.2e\n', tol);
Pl = Pl0;
Ql = Ql0;
Psp = Pg0 - Pl;
Qsp = Qg0 - Ql;
[V, del, converged, iter] = solve_power_flow(V, del, Psp, Qsp, Y, G, B, ...
                                              type, pq, nbus, tol,
max_iter);
if ~converged
    error('Base case pf did not converge!ab kia kareennn!');
end
fprintf('  bse case converged in %d iterations\n', iter);
fprintf('  initial point (lambda = %.4f)\n\n', lambda);
lambda_trace = [lambda_trace; lambda];
V_trace = [V_trace, V];
step_count = 0;
cpf_failed = false;
while lambda < lambda_max && ~cpf_failed
    step_count = step_count + 1;
    Pl = Pl0 + lambda * Pl_dir;

```

```

Ql = Ql0 + lambda * Ql_dir;

Psp = Pg0 - Pl;

Qsp = Qg0 - Ql;

J = compute_jacobian(V, del, G, B, nbus, pq);

dP_dlambda = -Pl_dir(2:end);

dQ_dlambda = -Ql_dir(pq);

F_lambda = [dP_dlambda; dQ_dlambda];

dx = J \ F_lambda;

t = [dx; 1];

t = t / norm(t);

d_theta_pred = t(1:nbus-1) * d_lambda;

d_V_pred = t(nbus:end-1) * d_lambda;

d_lambda_pred = t(end) * d_lambda;

del_pred = del;

del_pred(2:end) = del(2:end) + d_theta_pred;

V_pred = V;

V_pred(pq) = V(pq) + d_V_pred;

lambda_pred = lambda + d_lambda_pred;

Pl = Pl0 + lambda_pred * Pl_dir;

Ql = Ql0 + lambda_pred * Ql_dir;

Psp = Pg0 - Pl;

Qsp = Qg0 - Ql;

[V_corr, del_corr, converged, iter] = solve_power_flow(V_pred, del_pred,
...
Psp, Qsp, Y, G, B, type, pq, nbus,
tol, max_iter);

if ~converged

    fprintf(' Step %d: Corr failed at lambda = %.4f\n', step_count,
lambda_pred);

    fprintf(' reducing step s.....\n');

    d_lambda = d_lambda * 0.5;

```

```

        if d_lambda < 1e-4
            fprintf(' Step size too small. Stopping CPF.\n');
            cpf_failed = true;
        end
        continue;
    end

    V = V_corr;
    del = del_corr;
    lambda = lambda_pred;
    lambda_trace = [lambda_trace; lambda];
    V_trace = [V_trace, V];

    if mod(step_count, 5) == 0
        fprintf(' Step %3d: lambda = %.4f, Min V = %.4f pu (Bus %d)\n', ...
            step_count, lambda, min(V), find(V == min(V), 1));
    end

    % Adaptive step size control
    if iter <= 3
        d_lambda = min(d_lambda * 1.2, 0.1); % Increase step
    elseif iter >= 6
        d_lambda = d_lambda * 0.8; % Decrease step
    end
end

fprintf('-----\n');
fprintf('CPF completed: %d continuation steps\n', step_count);
fprintf('Final lambda: %.4f\n\n', lambda);
% Find the critical loading point (where stable region ends)
lambda_critical = lambda;
V_critical = V;
% Find voltage at critical buses
bus_14_critical_V = V_trace(14, end);

```

```

bus_3_critical_V = V_trace(3, end);

fprintf('  cpf comp \n');

fprintf('  Total continuation steps: %d\n', step_count);

fprintf('  Lambda range covered: [0, %.4f]\n', lambda);

fprintf('  Stable region explored successfully\n');

fprintf('    Bus 14 (Load Bus): %.4f pu\n', bus_14_critical_V);

fprintf('    Bus 3  (Weakest):  %.4f pu\n', bus_3_critical_V);

fprintf('    System Minimum:    %.4f pu at Bus %d\n', min(V_critical),
find(V_critical == min(V_critical), 1));

fprintf('  -----\n');

fprintf('  \n');

% Plot PV Curves

figure('Position', [100 100 1400 500]);

% Plot 1: Bus 14 PV Curve (Primary Result)

subplot(1, 3, 1);

hold on; grid on; box on;

plot(lambda_trace, V_trace(14, :), 'b-', 'LineWidth', 1.5);

xlabel('Loading Parameter  $\lambda$ ');

ylabel('Voltage Magnitude (pu)');

title('PV Curve - Bus 14');

% Plot 2: Bus 3 PV Curve (Weakest Bus)

subplot(1, 3, 2);

hold on; grid on; box on;

plot(lambda_trace, V_trace(3, :), 'r-', 'LineWidth', 1.5);

xlabel('Loading Parameter  $\lambda$ ');

ylabel('Voltage Magnitude (pu)');

title('PV Curve - Bus 3');

fprintf('At  $\lambda_{critical} = %.4f$  (End of Stable Region):\n\n',
lambda_critical);

fprintf('LOAD BUS VOLTAGES:\n');

fprintf('  Bus 14: %.4f pu\n', V_trace(14, end));

fprintf('  Bus 13: %.4f pu\n', V_trace(13, end));

fprintf('  Bus 12: %.4f pu\n', V_trace(12, end));

fprintf('  Bus 11: %.4f pu\n', V_trace(11, end));

```

```

fprintf(' Bus 10: %.4f pu\n', V_trace(10, end));
fprintf(' Bus 9: %.4f pu\n', V_trace(9, end));
fprintf(' Bus 5: %.4f pu\n', V_trace(5, end));
fprintf(' Bus 4: %.4f pu\n', V_trace(4, end));
fprintf(' Bus 3: %.4f pu (WEAKEST)\n\n', V_trace(3, end));

fprintf(' Minimum voltage: %.4f pu at Bus %d\n', min(V_critical),
find(V_critical == min(V_critical), 1));

fprintf(' Maximum voltage: %.4f pu at Bus %d\n', max(V_critical),
find(V_critical == max(V_critical), 1));

fprintf(' Average voltage: %.4f pu\n', mean(V_critical));

fprintf(' Voltage range: %.4f pu\n\n', max(V_critical) -
min(V_critical));

fprintf('loading info:\n');

fprintf(' Base case total load: %.2f MW\n', sum(P10) * BMva);

fprintf(' Critical point load: %.2f MW\n', sum(P10 + lambda_critical *
P1_dir) * BMva);

fprintf(' Load increase: %.2f MW (%.1f%%)\n', ...
sum(lambda_critical * P1_dir) * BMva, lambda_critical * 100);

fprintf(' Voltage stability margin: %.1f%% load increase\n',
lambda_critical * 100);

fprintf('=== cdf donne\n\n');

%% ===== HELPER FUNCTIONS =====

function [V, del, converged, iter] = solve_power_flow(V, del, Psp, Qsp, Y,
...
G, B, type, pq, nbus, tol, max_iter)

% Newton-Raphson power flow solver

converged = false;

iter = 0;

while iter < max_iter
    iter = iter + 1;

    % Calculate power injections

    P = zeros(nbus, 1);

    Q = zeros(nbus, 1);

```

```

    for i = 1:nbus
        for k = 1:nbus
            P(i) = P(i) + V(i) * V(k) * (G(i,k) * cos(del(i) - del(k)) +
...
                                                    B(i,k) * sin(del(i) -
del(k)));
            Q(i) = Q(i) + V(i) * V(k) * (G(i,k) * sin(del(i) - del(k)) -
...
                                                    B(i,k) * cos(del(i) -
del(k)));
        end
    end

    % Calculate mismatches
    dP = Psp - P;
    dQ = Qsp - Q;
    M = [dP(2:end); dQ(pq)];
    Tol = max(abs(M));

    if Tol < tol
        converged = true;
        break;
    end

    % Compute Jacobian
    J = compute_jacobian(V, del, G, B, nbus, pq);

    % Solve for corrections
    X = J \ M;

    % Update state variables
    del(2:end) = del(2:end) + X(1:nbus-1);
    V(pq) = V(pq) + X(nbus:end);
end

```

```

end

function J = compute_jacobian(V, del, G, B, nbus, pq)

    % Compute Jacobian matrix for power flow

    npq = length(pq);

    % J1: dP/d(delta)

    J1 = zeros(nbus - 1);

    for i = 1:nbus - 1
        m = i + 1;

        for k = 1:nbus - 1
            n = k + 1;

            if n == m

                for n2 = 1:nbus

                    J1(i, k) = J1(i, k) + V(m) * V(n2) * ...

                        (-G(m,n2) * sin(del(m) - del(n2)) + ...

                        B(m,n2) * cos(del(m) - del(n2)));

                end

                J1(i, k) = J1(i, k) - V(m)^2 * B(m, m);

            else

                J1(i, k) = V(m) * V(n) * ...

                    (G(m,n) * sin(del(m) - del(n)) - ...

                    B(m,n) * cos(del(m) - del(n)));

            end

        end

    end

end

% J2: dP/dV

J2 = zeros(nbus - 1, npq);

for i = 1:nbus - 1
    m = i + 1;

    for k = 1:npq
        n = pq(k);

        if n == m

```

```

        for n2 = 1:nbus
            J2(i, k) = J2(i, k) + V(n2) * ...
                (G(m,n2) * cos(del(m) - del(n2)) + ...
                B(m,n2) * sin(del(m) - del(n2)));

        end

        J2(i, k) = J2(i, k) + V(m) * G(m, m);

    else

        J2(i, k) = V(m) * ...
            (G(m,n) * cos(del(m) - del(n)) + ...
            B(m,n) * sin(del(m) - del(n)));

    end

end

end

end

% J3: dQ/d(delta)
J3 = zeros(npq, nbus - 1);

for i = 1:npq
    m = pq(i);

    for k = 1:nbus - 1
        n = k + 1;

        if n == m

            for n2 = 1:nbus
                J3(i, k) = J3(i, k) + V(m) * V(n2) * ...
                    (G(m,n2) * cos(del(m) - del(n2)) + ...
                    B(m,n2) * sin(del(m) - del(n2)));

            end

            J3(i, k) = J3(i, k) - V(m)^2 * G(m, m);

        else

            J3(i, k) = V(m) * V(n) * ...
                (-G(m,n) * cos(del(m) - del(n)) - ...
                B(m,n) * sin(del(m) - del(n)));

        end

    end

end
end

```

```

end

% J4: dQ/dV
J4 = zeros(npq);
for i = 1:npq
    m = pq(i);
    for k = 1:npq
        n = pq(k);
        if n == m
            for n2 = 1:nbus
                J4(i, k) = J4(i, k) + V(n2) * ...
                    (G(m,n2) * sin(del(m) - del(n2)) - ...
                    B(m,n2) * cos(del(m) - del(n2)));
            end
            J4(i, k) = J4(i, k) - V(m) * B(m, m);
        else
            J4(i, k) = V(m) * ...
                (G(m,n) * sin(del(m) - del(n)) - ...
                B(m,n) * cos(del(m) - del(n)));
        end
    end
end
end

% Assemble full Jacobian
J = [J1 J2; J3 J4];
end

```

RESULTS

PV curve plots for one or more buses:

